

Шакиров Тимур Эдуардович

Инженер. Языковые модели в продакшене: шлюзы, биллинг, конвейеры контента. Последний год - e-commerce платформы на React и GraphQL.

Санкт-Петербург · удалённо +7 981 691-94-36 edtimyr@gmail.com
github.com/TimaxLacs github.com/TimaxHack github.com/timaxlax npmjs.com/~timax

Работа распределена между тремя аккаунтами: основной объём кода в deep-assistant написан под TimaxHack.

НАВЫКИ

ЯЗЫКИ	Python, TypeScript, JavaScript
ФРОНТЕНД	React, Next.js, Vite, Tailwind, Capacitor (сборки под iOS и Android)
БЭКЕНД	Node.js, Express, FastAPI, GraphQL (Hasura), REST, PostgreSQL / PostGIS, Redis, MongoDB, SQLite, MinIO
ИИ	маршрутизация между провайдерами и фолбэк, биллинг токенов, RAG и MCP, очереди генерации текста и изображений, reasoning-конвейеры, распознавание и синтез речи, LangChain
ПРОД	Docker, nginx, systemd, Linux, Git, CI, настройка DNS и почты

ОПЫТ РАБОТЫ

Deep.Assistant - агрегатор нейросетей

2024-2025

Сооснователь, backend и DevOps · команда из трёх человек

Коммерческий продукт: чат с моделями, генерация изображений, музыки и голоса, API-шлюз, мини-приложения в Telegram и VK.

LLM-шлюз - github.com/deep-assistant/api-gateway

Спроектировал и написал первую версию; вырос из моего личного репозитория resale-chatgpt-azure в инфраструктуру, через которую идут все запросы продукта. Основной автор кода: HTTP-сервер, конфигурация моделей, менеджер токенов, слой работы с базой.

- Около 37 моделей от шести поставщиков за одним OpenAI-совместимым интерфейсом: OpenAI напрямую, OpenRouter, DeepInfra, DeepSeek и реселлеры.
- Таблица деградации вместо простого ретрая.** Для каждой модели прописана упорядоченная цепочка из 12-16 замен: если провайдер недоступен, запрос уходит на ближайшую по смыслу модель, а не просто падает. Замены подбирал вручную, чтобы ответ оставался пригодным для того же сценария.
- Единый счёт поверх несопоставимых прайсов.** У каждой модели свой коэффициент пересчёта токенов во внутреннюю валюту - от 1.7 у крупных моделей до 40 у мелких. Пользователь видит один баланс, продукт считает себестоимость запроса и наценку.
- Переводы между счётами без СУБД.** Хранилище файловое, поэтому сделал эксклюзивные блокировки с распознаванием зависших, запись через временный файл с обратной вычиткой и ротацией резервных копий, а также восстановление незавершённых переводов при старте.
- Авторизация по ключам, общий баланс для бота и внешнего API.

Веду репозиторий: принимаю и ревьюю изменения второго разработчика.

Telegram-бот - github.com/deep-assistant/telegram-bot

Мультимодельный ассистент на aiogram, над которым работали четверо. Мои части:

- **Переводы внутренней валюты между пользователями.** Диалог на конечном автомате: получатель, сумма, подтверждение, отмена, история операций. Модуль писал с нуля.
- Сервис синхронизации пользователей: ленивая догрузка профиля при первом обращении и массовая сверка с ограничением параллелизма, чтобы не упереться в лимиты Telegram.
- Режим транскрибации аудио с отдельным коэффициентом тарификации и пользовательские системные сообщения.
- Поведение в групповых чатах: бот отвечает только на обращения к себе, разбирает упоминания в подписях к фото и документам. Выгрузка истории диалога файлом.
- Оплата через Telegram Stars и интеграция внешнего рекламного API за фича-флагом.
- Разделение окружений TEST и PROD, docker-compose под оба, асинхронные проверки доступности провайдеров.

Бот @DeepGPTBot, канал @gptDeep - 1 553 подписчика.

Продуктовая работа

Завёл 14 продуктовых задач по боту: умный приоритет провайдеров, режим экспресс-ответов, напоминания, сохранение прошлых контекстов, подключение новых моделей. Задачу про умный приоритет API коллега реализовал через неделю.

E-commerce платформа (проект заказчика, под NDA)

декабрь 2025 - настоящее время

Контрактный инженер: фронтенд, гео, мобильные сборки, смежный DevOps

Маркетплейс с каталогом и картами: монорепозитарий фронтенда и восьми backend-сервисов. React и TypeScript, Hasura GraphQL, PostgreSQL с PostGIS, Redis, MinIO, nginx, Docker, Capacitor. Веду клиентское приложение, участвую в проектировании архитектуры, на бэкенде отвечаю за платежи, баланс и обработчики событий, держу серверную часть. Детали продукта под NDA, поэтому ниже - модули и решения.

Витрина, каталог и поиск

Каталог, карточка товара, редактирование позиций продавцом, подборка на главной. Поиск с автодополнением, фильтры и разбор поисковой строки, библиотека UI-компонентов, на которой собран весь интерфейс.

Регистрация, вход и профиль

Несколько типов аккаунтов со своими схемами валидации, сброс пароля, многофакторное подтверждение, серверное хранилище регистрационных токенов, профиль с загрузкой аватара.

Двуязычность и автоперевод карточек

Интерфейс на русском и английском: словари вынесены из компонентов и типизированы. Перевод карточек не ручной - хук автоперевода, серверный роут и CLI-скрипт, прогоняющий каталог пачкой. Новый язык не требует правки разметки.

Кошелёк, оплата и заказы

Вертикаль от интерфейса до базы: сервисы баланса и платежей, роуты покупок и ставок, курс валют, внешний эквайринг, кошелёк, оформление и страница заказа. Смена статуса сделки и новые отклики идут через событийные обработчики.

Мобильные приложения на Capacitor

Из той же кодовой базы собираются приложения под iOS и Android: проекты Xcode и Gradle, доступ к геолокации, платформенная навигация, определение платформы в рантайме, сервис-воркер для офлайн-кеша. Прошёл требования App Store: манифест приватности и механика жалоб и блокировок для пользовательского контента - без неё приложение с UGC не публикуют.

Конвейер наполнения каталога

Спроектировал схему загрузки товаров от внешних поставщиков: источники за общим контрактом, дедупликация по внешнему идентификатору, штрихкоду и отпечатку характеристик.

Нейросети работают *после* сбора данных, отдельными очередями - воркер описаний RU и EN и воркер изображений. Сбор не блокируется генерацией, упавший воркер оставляет товар на витрине с исходными характеристиками. Характеристики поставщика модель не переписывает.

Гео-модуль - отдельный продукт внутри платформы

Модуль на 11 700 строк, который вёл от начала до конца. Единый интерфейс поверх трёх картографических провайдеров: Яндекс, Google и 2ГИС. Архитектура на адаптерах - компоненты карты, маркера и зоны ничего не знают о конкретном провайдере; движки переадресуют команду адаптеру, а тот переводит унифицированные данные в вызовы своего SDK. Смена провайдера - это смена одного параметра.

- **Геометрия зон, около 1 700 строк:** буферные зоны вдоль маршрута с плавными переходами к круговым зонам маркеров через квадратичные кривые Безье, объединение полигонов с поддержкой отверстий и мультиполигонов, несколько режимов объединения, валидация геометрии и кэширование расчётов.
- Перетаскиваемые маркеры и пользовательские зоны; геокодинг вынесен на серверный роут, чтобы ключи API не попадали в браузер.
- Хранение геометрии в PostGIS, поиск по радиусу, смюк-тесты и описание того, что нужно вынести для запуска модуля отдельным приложением.

Сервер и выкладка

Развёртывание с нуля по написанной инструкции: systemd для backend-сервисов, политика перезапуска контейнеров, nginx на фронте. Провёл миграцию базы на PostGIS - без неё пространственные запросы упирались в перебор на стороне приложения. Держу сервер и разбираю сбои.

ManuAir - платформа международных денежных переводов

август 2025 - май 2026

Проект заказчика: фронтенд и смежный бэкенд

Микросервисная система вокруг Hasura GraphQL: router, обработчик событий, рассылка, Telegram-бот, генератор документов, курсы валют; PostgreSQL, Redis, MinIO.

- **Устойчивость доставки уведомлений:** при недоступном канале статус возвращает «не отправлено», а приглашение не помечается доставленным.
- **Изоляция данных между ролями:** сброс кеша запросов при смене пользователя, ограничение подписок на детали сделки.
- Вывод из системы устаревшего домена приглашений: таблицы, права Hasura, роуты, обработчики, сгенерированные типы.
- Обслуживание серверной части: выкладка сервисов, разбор сбоев, настройка почты и DNS проекта.

Matreshka - AI/e-commerce платформа

2026

Проект заказчика: фронтенд и запуск публичного сайта · matreshka.club

Публичный сайт платформы: React, Vite, TypeScript, Tailwind, фирменный стиль. Выложил на nginx с SSL, настроил DNS и почту на домене. Второй проект для того же заказчика.

Fullstack, заказная разработка · taxi628.ru

Telegram-бот биржи заказов для таксопарка: оператор размещает заказ, водители видят и подтверждают его в реальном времени. Внедрён у заказчика, заменил ручную раздачу смен.

ЭКОСИСТЕМА DEEP.FOUNDATION

Ассоциативная модель данных: участие с 2024 года

2024-2025

github.com/deep-foundation · участник организации

Deep.Foundation строит хранилище, где единственная сущность – связь: кортеж из идентификатора, типа и ссылок на другие связи. Связь может ссылаться на связь, поэтому одной таблицей описываются и данные, и схема, и обработчики. Я пришёл туда как пользователь платформы и остался писать инструменты вокруг неё.

- **deep-files-sync - проекция графа связей на файловую систему.** Мой пакет, 13 версий. Связи раскладываются в каталоги, значение лежит в текстовом файле рядом с объектом, а вложенность папок выражает отношения: путь по директориям изоморфен запросу к базе. Правки с обеих сторон синхронизируются, подписка на изменения материализуется файлом, удаление файла означает отписку. Смысл в том, чтобы содержимое базы можно было читать и править обычными инструментами – `grep`, редактором, `git`. github.com/TimaxLacs/deep-files-sync
- **Пакеты ИИ-тулинга в официальном скоупе.** Выпускал версии в `@deep-foundation/ai`, `ai-models`, `ai-templates`, `ai-tasks`, `4o-bot`, `chatgpt-azure-deep`, `function_generation`, `tests_generation`. Такой пакет – не библиотека, а выгрузка настроенного графа связей, которая разворачивается в чужом инстансе.
- **Диалоги бота как граф.** Реализовал запись переписки в ассоциативное хранилище: диалог, сообщение, автор и ответ выражены типами связей, без отдельных таблиц под каждую сущность. Решение прожило в проде около шести недель и было заменено на файловое хранилище – сложность не окупилась.
- Соавтор русскоязычной теоретической документации по модели, около 1 000 строк. Завёл порядка 36 issue – в основном баг-репорты по развёртыванию платформы, с трейсбеками.

ИНЖЕНЕРНЫЕ КЕЙСЫ С ИИ

Конвейеры и агентные процессы

2025-2026

- **test-resorning** – reasoning-конвейер на моделях без встроенного режима рассуждений: предсказание намерения, пошаговое решение, состязательная критика собственного ответа, доработка, синтез. Вариант на LangChain с Tree of Thoughts и самосогласованностью. Апрель 2025, до того как такой режим стал типовой функцией.
- **ai-post-generator** – контент-конвейер с отдельной ИИ-модерацией: черновик, проверка вторым вызовом, до трёх кругов правок, публикация. Состояние в SQLite, чтобы пережить падение процесса; публикаторы под Telegram, VK и MAX вынесены в стратегии.
- **cursor-agent-workflows** – набор агентных процессов на 1 900 строк спецификаций: конвейер безопасного изменения кода из пяти агентов с запечатанными границами задачи и параллельными потоками, бутстрап проекта, процесс исследования. Пользуюсь ежедневно.

Голосовой стек

2025

github.com/TimaxLacs/InnerEcho

Голосовой ассистент целиком на локальной машине: распознавание речи, языковая модель, синтез с клонированием голоса, автонастройка аудиоустройств. Отдельно сравнивал модели синтеза речи на

русском корпусе. Исследовательский прототип, не прод.

ОТКРЫТЫЙ КОД И ПАКЕТЫ

Пакеты синтеза речи и утилиты

2024-2025

npmjs.com/~timax

- **xtts-v2-js** - обёртка над XTTS-v2 с клонированием голоса на 17 языках: сама поднимает Python-окружение, чтобы пакет ставился как обычная зависимость Node.js. 7 версий.
 - **zonosjs** и **zonos-client** - клиент к локальному серверу синтеза речи Zonos, 10 версий суммарно.
 - **@timax/passport-authorization** и связанные пакеты - регистрация и подтверждение по email.
-

tg-runner - оркестрация ботов

январь 2026

Комплект из четырёх компонентов

Оркестратор запуска ботов на конечном автомате с Redis, воркер, поднимающий ботов в Docker, публикуемый SDK для своих воркеров и CLI. Docker Compose, pytest, документация.

github.com/TimaxLacs/ - [tg-runner-orchestrator](#) · [tg-runner-worker](#) · [tg-runner-worker-sdk](#) · [tg-runner-cli](#)

ПРОЕКТЫ

Боты для мессенджера MAX

2026

github.com/TimaxLacs/max-bot

Мультимодальная обработка фото нейросетью, квота на пользователя, Docker, интеграция с асинхронным ИИ-воркером и запасной сценарий при недоступности бэкенда.

vk-tg-parser

2026

github.com/TimaxLacs/vk-tg-parser

pip-пакет: единый парсер Telegram через MTPROTO и VK через API с общей моделью данных и мониторингом в реальном времени.

Автокомментер по хэштегам

2026

github.com/TimaxLacs/tg-hashtag-commenter

Комментирование по хэштегам в Telegram и VK с ограничением частоты и дедупликацией. Реальный прогон в мае 2026.

ai-lector

2025

github.com/TimaxLacs/ai-lector

Бот, перерабатывающий аудиолекции под нужный формат. Собрал локальный Telegram Bot API server через stake, чтобы обойти лимит на размер файла.

anti-spam-ai

2025

github.com/TimaxHack/anti-spam-ai

Антиспам-бот: модель через OpenRouter выставляет сообщению спам-балл от 1 до 10.

github.com/TimaxLacs/Orbiteer

Анализатор рисков космических миссий: расчёт вероятности столкновений, 3D-визуализация орбит на three.js, API на Sanic.

РАННИЕ РАБОТЫ, ДЛЯ КОНТЕКСТА

GEFFEST - перчатка-манипулятор

2021-2022

github.com/TimaxHack/GEFFEST

Перчатка, которая повторяет движения руки на манипуляторе. Положение пальцев и кисти снимается с датчиков и считается через кватернионы - это убирает срывы, которые дают углы Эйлера при переворотах кисти. Перчатка и манипулятор общаются по радио собственным компактным протоколом, поэтому задержка не растёт с числом каналов.

На стороне манипулятора движение сервоприводов сглаживается, иначе дрожь руки переходит в рывки механики. Схему и макетные платы паял сам. Отдельно написал Android-приложение голосового управления на Java и Vosk - распознавание работает без интернета.

Ранние боты и компьютерное зрение

2021

Бот-библиотека - карта библиотек с нужной книгой, командный проект. Бот замерзания Онежского озера - определение уровня замерзания по кадрам с уличной камеры компьютерным зрением.

Запуски, которые не взлетели

2023-2025

ArtApollo - студия, где заказы отрисовывают нейросети. Собрал команду до 30 человек, взял первые платные заказы, выполнил корпоративный заказ на 12 человек. Проект закрыт, команда распалась.

Cyber Vestis - прототип умной одежды с климат-контролем: ПО для сенсорного дисплея, телеметрия температуры тела. Дошли до рабочего прототипа, до инвестиций не дошли.

Обе истории закончились ничем, но дали опыт сборки команды и доведения прототипа до демонстрации.
